

Creación de una CLI moderna con comandos

slash

para agentes de IA

Ejemplos de CLI modernas (Anthropic, OpenAI, Google)

Las compañías líderes en IA han desarrollado herramientas de línea de comandos (CLI) interactivas para desarrollar con asistencia de IA. Por ejemplo, Anthropic Claude Code, OpenAI Codex CLI y Google Gemini CLI son entornos de terminal donde un agente de IA ayuda con tareas de programación y automatización. Estas CLI ofrecen una experiencia de uso “moderna” dentro del terminal, con interfaz interactiva, texto con formato enriquecido y comandos especiales para controlar la sesión. OpenAI, por ejemplo, lanzó Codex CLI como un agente de código local, de código abierto y escrito en Rust para maximizar la velocidad y eficiencia . Anthropic y Google distribuyen sus CLIs como paquetes de Node.js (instalables vía npm), lo que indica que internamente usan TypeScript/JavaScript sobre Node para portabilidad y buen rendimiento . Todas estas herramientas funcionan localmente en tu máquina, accediendo a modelos de IA en la nube o locales según la configuración, pero con la comodidad de un entorno tipo chat en la terminal. En el caso de Codex CLI, “corre en una interfaz de terminal interactiva. Puedes pedirle cualquier cosa y el agente leerá tu código, hará ediciones y ejecutará comandos en tu proyecto” . Esto permite comunicarse con el modelo de IA de forma conversacional y que este realice acciones útiles sobre el código fuente u otras tareas de desarrollo.

Comandos

slash

: atajos para instrucciones y tareas frecuentes

Muchas de estas CLI incorporan comandos “slash” (/comandos/), inspirados en interfaces de chat modernas, para lograr una comunicación más fluida con el agente de IA. En lugar de tener que escribir largas instrucciones o recordar secuencias complejas, los comandos slash funcionan como atajos predefinidos: comienzan con una barra diagonal / seguida del

nombre de la acción, y ejecutan tareas o prompts específicos. “En Claude Code, los comandos slash son básicamente atajos que escribes en el chat, comenzando con ‘/’, para ejecutar tareas predefinidas”, en vez de redactar un prompt extenso cada vez que quieres revisar el código o generar un componente nuevo . Su objetivo principal es simplificar y agilizar las interacciones repetitivas, ayudándote a: automatizar tareas comunes con un solo comando, mantener procesos consistentes en tu equipo, y “usar menos tokens: las instrucciones complejas consumen mucho espacio en la ventana de contexto; al guardarlas en un comando externo, ahorras tokens y mantienes el chat enfocado” . En resumen, los comandos slash actúan como “hechizos personalizados” que encapsulan flujos de trabajo habituales en una sola línea .

Hay comandos slash integrados (built-in) y comandos personalizados. Los integrados vienen con la herramienta para gestionar la sesión o configuración del agente, por ejemplo: `/help` (muestra la ayuda con todos los comandos disponibles), `/clear` (reinicia el historial de la conversación), `/model` (cambia el modelo de IA activo) o `/login` (autentica tu cuenta en el servicio) . Estos comandos internos sirven para controlar el comportamiento del agente y la interfaz (p.ej. limpiar el contexto, ajustar permisos, ver uso de tokens, etc.). En cambio, los comandos slash personalizados son definidos por el usuario o el proyecto para realizar tareas específicas de desarrollo o automatizar pasos frecuentes. Por ejemplo, podrías crear comandos `/deploy`, `/run-tests` o `/new-component` adaptados a tu proyecto, que en una sola orden ejecutan todos los pasos necesarios (compilar, testear, desplegar, etc.) . Estos comandos los diseñas tú mismo – totalmente personalizables en prompt y lógica – normalmente mediante archivos de definición en tu repositorio o en tu directorio personal.

¿Cómo se crean estos comandos personalizados? Las CLI modernas lo hacen de forma muy sencilla: en Anthropic Claude Code, basta con crear archivos Markdown con el prompt deseado. Por ejemplo, si en tu repositorio añades `.claude/commands/optimize.md` con el contenido “Analyze this code for performance issues and suggest optimizations:”, automáticamente tendrás un comando `/optimize` disponible en esa sesión (marcado como comando de proyecto) . De igual manera, si creas `~/.claude/commands/security.md` en tu home con el texto “Review this code for security vulnerabilities:”, dispondrás de `/security` en cualquier proyecto (como comando personal del usuario) . Al invocar `/optimize` o `/security` en la conversación, el sistema tomará esas plantillas de prompt y se las enviará al modelo de IA junto con el contenido de código que corresponda, sin que tengas que escribir la instrucción completa cada vez.

Además, estos atajos aceptan argumentos dinámicos para mayor flexibilidad. Por ejemplo, supongamos que definimos un archivo de comando `fix-issue.md` con el contenido: “Fix issue `#$ARGUMENTS` siguiendo nuestros estándares de código” . El marcador especial `$ARGUMENTS` será reemplazado por lo que escribas después del nombre del comando. Así, si en la sesión escribes `/fix-issue 123`, la CLI insertará “123” en la plantilla y enviará al agente la instrucción completa para que “revise y solucione el bug del issue #123”. Este mecanismo de argumentos (como `$ARGUMENTS`, o incluso `$1`, `$2` para parámetros posicionales) permite que un solo comando slash sirva para múltiples variaciones de una tarea .

En el caso de Google Gemini CLI, el concepto es muy similar aunque el formato de definición difiere: se usan archivos `.toml` en lugar de Markdown. Un archivo TOML incluye al

menos una descripción y un campo prompt. El nombre del archivo TOML define el nombre del comando, y dentro puedes usar plantillas como `{{args}}` para insertar los argumentos proporcionados al comando, o incluso ejecutar comandos del shell local e incrustar su salida en el prompt usando sintaxis especial (`!{comando}`). Por ejemplo, Gemini CLI proporciona en su documentación un comando `/review <número>` que al ejecutarse toma el número de PR de GitHub indicado, corre internamente comandos como `gh pr view` y `gh pr diff` para obtener detalles y diffs del pull request, y luego envía al modelo un prompt que integra toda esa información para generar un código de revisión detallado. Esto muestra el potencial de los slash commands: pueden no solo contener texto estático, sino interactuar con el entorno (leer archivos, obtener diffs, resultados de tests, etc.) y pasar ese contexto al agente de IA de forma transparente para el usuario. En todos los casos, la CLI se encarga de interpretar el comando slash ingresado, reunir la información necesaria (ya sea texto fijo de un archivo, argumentos del usuario, o resultados de herramientas externas) y luego llamar al modelo de IA con la solicitud resultante. Para el usuario, la experiencia es muy fluida: con una línea estilo `/fix-issue 123` o `/review 42`, desencadena un diálogo con la IA que realiza una tarea compleja sin necesidad de escribir pasos detallados manualmente.

Implementación de una CLI local con estas características

Para crear una CLI moderna de este tipo en tus propios proyectos (incluso para interactuar con tu agente memtech), debes planificar tanto la tecnología a usar como el diseño de la interacción. Un factor clave es escoger el lenguaje y entorno adecuados. Idealmente, quieres un lenguaje que sea popular y con buen soporte de ecosistema (para facilitar integraciones con Python, Java, TypeScript, etc.) y que ofrezca buen rendimiento para una experiencia interactiva ágil. Hay varias rutas posibles:

- **Node.js / TypeScript:** Es una opción muy popular para CLIs actuales. Tanto la CLI de Anthropic como la de Google se distribuyen vía npm, lo que sugiere que están escritas en TypeScript/Node (fácil de instalar en cualquier sistema con Node). Node es eficiente, multi-plataforma y puede invocar procesos externos (por ejemplo, ejecutar un script Python o un jar de Java) fácilmente con funciones del módulo `child_process`. Además, con TypeScript se obtiene tipado que ayuda a mantener un código ordenado a medida que la herramienta crece. Existen librerías como `Commander.js`, `Inquirer/Enquirer` o incluso `oclif` que facilitan la construcción de CLIs en Node. Para colores y formato de texto, se puede usar `chalk` u otras librerías, aunque muchas veces puedes formatear usando códigos ANSI manualmente.
- **Python:** Es otra gran opción, sobre todo si tu agente de IA o sus integraciones ya están en Python. Python tiene excelentes bibliotecas para CLI, como `Click` o `Typer` (para parseo de argumentos y comandos de manera declarativa) y también `Prompt Toolkit` (para construir REPL interactivos con autocompletar, historial, etc.). Si quieres una apariencia más pulida, la biblioteca `Rich` permite agregar fácilmente colores, estilos y contenido enriquecido en la terminal – “Rich es una librería de Python para texto enriquecido y formato bonito en la terminal. Su API facilita añadir color y estilo a la salida”. Con `Rich` (y su extensión `Textual`) incluso puedes crear interfaces semi-gráficas (como tablas, paneles, barras de progreso, resaltado de sintaxis de

código, etc.) para lograr esa “visual moderna” en tu CLI. Python tiene la ventaja de la simplicidad y de contar con multitud de librerías para interactuar con APIs de IA, archivos, herramientas dev, etc., aunque podría ser un poco más lento que otras opciones al iniciar. En muchos casos esto no es crítico, ya que el tiempo dominante será el de las llamadas al modelo de IA, no el overhead del CLI en sí.

- Lenguajes compilados (Go, Rust): Si “rápido de ejecutar” es una prioridad absoluta y buscas distribuir un binario único sin dependencias, podrías considerar usar Go o Rust. De hecho, OpenAI eligió Rust para su Codex CLI “por velocidad y eficiencia” . Rust ofrece gran rendimiento y control de bajo nivel, y hay crates para construir TUIs (por ejemplo, crossterm o tui-rs) que permiten hacer interfaces muy ricas en terminal. Go, por su parte, es excelente para CLIs simples, con librerías como Cobra/Viper para comandos. Ambos permiten fácilmente llamar a procesos del sistema (por ejemplo, usar `exec` en Go o `std::process::Command` en Rust para invocar Python, Git u otras herramientas). La desventaja es que el desarrollo puede ser más complejo que en Python/JS, y el ecosistema de IA en Rust/Go es más limitado – aunque siempre puedes usarlos solo como shell que llama a un backend en Python u otro lenguaje.

En la práctica, una arquitectura común es que la CLI actúe como orquestador local: maneja la interacción con el usuario, parsea los comandos slash, y cuando se trata de ejecutar una tarea AI envía la solicitud a un agente de IA (ya sea vía API HTTP, vía llamada a una librería local, o incluso a un proceso servidor local). Por ejemplo, tu CLI podría consumir la API de OpenAI/Anthropic u otro LLM para obtener respuestas. En tu caso, si dispones de un agente memtech propio, podrías exponerlo como una librería Python (si es Python) o un servicio local, y la CLI se comunica con él. Asegúrate de diseñar una interfaz clara: cuando el usuario ingresa texto normal (no comenzando con /), la CLI lo envía como mensaje al agente de IA; cuando ingresa un comando /..., la CLI lo intercepta y no se lo pasa directamente al modelo sino que ejecuta la lógica correspondiente (por ejemplo, `/clear` limpiará el contexto conversacional almacenado, `/save` podría guardar el historial a disco, `/optimize` cargará un prompt predefinido y lo enviará al modelo, etc.). Este enrutamiento puedes implementarlo con un simple parser de comandos: verifica si la línea empieza con '/', separa el comando y sus argumentos, y luego haz un match/case para cada comando soportado. Para comandos personalizados, podrías, por ejemplo, buscar un archivo `.miagente/commands/<nombre>.txt/md` y utilizar su contenido como prompt.

No olvides también incluir un comando `/help` que liste todos los comandos disponibles (como hacen Claude y Gemini CLI), para que el usuario descubra fácilmente cómo interactuar. Y en cuanto a la persistencia de contexto, es útil que la CLI mantenga un historial de la conversación con el agente de IA para soportar interacciones multi-turno. Herramientas como Claude Code y Codex CLI mantienen el historial de mensajes (preguntas y respuestas) en memoria e incluso ofrecen comandos para visualizarlo o retroceder. Tú podrías implementar una simple lista de mensajes previos que se envían junto con cada nueva pregunta al modelo (respetando los límites de tokens). El comando `/clear` en tu CLI simplemente vaciaría esa lista para empezar una sesión fresca .

En términos de visualización moderna en el terminal: utiliza colores para diferenciar la salida del agente (por ejemplo, mostrando las respuestas de la IA en cian o verde) y el texto del usuario en otro color o estilo, agrega formatos como subrayado o negrita para títulos, resalta el código que devuelva el agente (quizá con sintaxis resaltada), etc. Las CLI oficiales suelen tener un prompt especial (ej. > o algún emoji) para las entradas del usuario, y quizás un prefijo para las respuestas de la IA. Algunas incluso muestran un indicador de “pensando...” mientras el modelo procesa. Todos estos detalles de UX hacen la interacción más cómoda. Puedes inspirarte en la estética minimalista pero funcional que presentan estas herramientas de Anthropic/OpenAI: por ejemplo, Claude Code muestra una línea de estado con el modelo activo, el consumo de tokens, etc., y permite editar configuraciones en la marcha con menús interactivos. Implementar algo así puede ser avanzado, pero podrías empezar con elementos básicos e ir refinando.

Por último, prueba tu CLI en diferentes entornos (Linux, Windows, Mac) para garantizar que el manejo de entrada/salida funciona correctamente en todos (especialmente si usas secuencias ANSI para colores o dependes de comandos shell). Si es local y planeas usarlo solo tú o tu equipo, puedes ajustarla a tus necesidades específicas sin problema. Pero si planeas liberarla públicamente, considera ofrecer facilidades de instalación (como hace Google con npx @google/gemini-cli o OpenAI con Homebrew) y una documentación clara.

Conclusión

Construir una CLI moderna para interactuar con agentes de IA implica combinar una buena experiencia de usuario en la terminal con la potencia de los LLMs. Los ejemplos de Anthropic, OpenAI y Google nos muestran la ruta: proporcionar una interfaz de conversación fluida, enriquecida con comandos slash que actúan como atajos para tareas frecuentes, evitando la necesidad de redactar comandos extensos en cada ocasión. Aprovechando un lenguaje adecuado (Python/TypeScript para rapidez de desarrollo, o Rust/Go para máximo rendimiento) y las librerías disponibles, puedes crear una herramienta de terminal local que te permita comunicarte con tu agente IA de forma natural y eficiente. De esta manera, tu CLI puede convertirse en un copiloto que asiste en tu flujo de trabajo diario: desde analizar código, generar funciones o configuraciones, hasta automatizar despliegues, todo mediante comandos cortos y conversacionales, con la lógica compleja delegada al agente de inteligencia artificial en segundo plano. ¡Manos a la obra con esos slash commands! 🚀

Fuentes: Las características y ejemplos mencionados se basan en la documentación oficial de Claude Code (Anthropic), Gemini CLI (Google) y Codex CLI (OpenAI), así como en guías de terceros sobre mejores prácticas en comandos slash. Estas referencias detallan cómo los comandos slash ayudan a agilizar el trabajo con IA en la terminal y cómo se pueden crear comandos personalizados para adaptarse a distintas necesidades. Hemos tomado inspiración de dichas fuentes para proporcionar una visión integrada de cómo desarrollar tu propia CLI inteligente.